ICONIX

JC.

Introducción

Iconix es una metodología pesada-ligera de Desarrollo del Software que se halla a medio camino entre un RUP (Rational Unified Process) y un XP (eXtreme Programming).

Iconix deriva directamente del RUP y su fundamento es el hecho de que un 80% de los casos pueden ser resueltos tan solo con un uso del 20% del UML, con lo cual se simplifica muchísimo el proceso sin perder documentación al dejar solo aquello que es necesario. Esto implica un uso dinámico del UML de tal forma que siempre se pueden utilizar otros diagramas además de los ya estipulados si se cree conveniente.

Iconix se guía a través de casos de uso y sigue un ciclo de vida iterativo e incremental. El objetivo es que a partir de los casos de uso se obtenga el sistema final.

Característica ICONIX

Iterativo e incremental: Varias iteraciones ocurren entre el desarrollo del modelo del dominio y la identificación de los casos de uso. El modelo estático es incrementalmente refinado por los modelos dinámicos.

Trazabilidad: Cada paso está referenciado por algún requisito. Se define trazabilidad como la capacidad de seguir una relación entre los diferentes "artefactos de software" producidos.

Dinámica del UML: La metodología ofrece un uso "dinámico" del UML por que utiliza algunos diagramas del UML, sin exigir la utilización de todos, como en el caso de RUP o PUDS.

La página {p} está borrosa: ¡Haga clic en "Desbloquear Scribd" en la extensión BamiDoc antes de descargar!

Etapas de ICONIX

Nota: Lo que se encuentra con fuente azul y subrayada en cada fase es el documento que se genera en esa fase.

1.- Análisis de Requisitos

A) Identificar en el "mundo real", los objetos y todas las relaciones de agregación y generalización entre ellos, Utilizar un diagrama de clases de alto nivel definido como **modelo de dominio**

B) Presentar, si es posible, una **prototipacion rápida** de las interfaces del

Desbloquea este documento
Carga un documento para descargarlo o suscríbete para leerlo y descargarlo.
Cargar para descargar
Comienza tu prueba gratuita de 30 días
O
Desbloquea las próximas 3 páginas después de un anuncio
Omitir anuncio

C) Identificar los casos de uso del sistema mostrando los actores involucrados (En esta parte identificamos las funcionalidades del sistema en general), Utilizar para representarlo el **modelo de casos de uso**

D) Organizar los casos de uso en grupos o módulos, utiliza el **diagrama de paquetes**

E) Identificar requisitos funcionales y no funcionales a partir de los casos de uso y marcar la trazabilidad de esos. Esto se documenta con una **lista de requisitos funcionales y no funcionales**

Nota: Un importante aspecto de ICONIX es que un requisito se distingue explícitamente de un caso de uso, en este sentido un caso de uso describe un comportamiento o funcionalidad, un requisito describe una regla para este comportamiento.

1.- Análisis y Diseño Preliminar

A) Describir los casos de uso, como un flujo principal de acciones, pudiendo contener los flujos alternativos y los flujos de excepción. La principal sugerencia de ICONIX en esta actividad, es que no se debe perder mucho tiempo con la descripción textual, Debería usarse un estilo consistente que sea adecuado al contexto del proyecto, para documentar esta parte se utiliza el documento de **especificación de casos de uso**

B) Realizar un **diagrama de robustez**. Se debe ilustrar gráficamente las interacciones entre los objetos participantes de un caso de uso, este diagrama permite analizar el texto narrativo de cada caso de uso e identificar un conjunto inicial de objetos participantes de cada caso de uso,

C) Actualizar el **diagrama de clases** ya definido en el modelo de dominio con las nuevas clases y atributos, descubiertas en el diagrama de robustez

3.- Diseño

A) Especificar el comportamiento a través del **diagrama de secuencia**, para cada caso de uso identificar los mensajes entre los diferentes objetos.

B) Terminar el modelo estático, adicionando los detalles del diseño en el **diagrama de clases final**.

C) Verificar si el diseño satisface todos los requisitos identificados, se utiliza un **checklist de los requisitos**

4.- Implementación

A) Utilizar el **diagrama de componentes**, si fuera necesario para apoyar el desarrollo. Es decir, mostrar la distribución física de los elementos que componen la estructura interna del sistema.

B) Escribir / Generar código. Debemos tomar en cuenta factores como :

- **a.** La reusabilidad, que es la posibilidad de hacer eso de los componentes en diferentes aplicaciones.
- **b.** La extensibilidad, que consiste en modificar con facilidad el software
- **c.** La confiabilidad, realización de sistemas descartando las posibilidades de error.

C) Realizar pruebas. Test de unidades, de casos, datos y resultados. Test de integración con los usuarios para verificar la aceptación de resultados. Las pruebas también deben ir documentadas, se podría utilizar el estándar **IEEE 829** o el método **MECAP**, para documentar.